Timeformer: Explicit Temporal Conditioning for Traceable Semantic Representations

Anonymous Author

Institution withheld for review

Abstract. Words change meaning over time, yet most computational models treat token representations as period-independent. We study *temporal traceability*: whether the same surface token, queried at different periods, occupies period-appropriate neighborhoods. A model may improve predictive accuracy with temporal information while still assigning nearly identical neighborhoods across periods; this gap is our focus.
We introduce Timeformer and evaluate it through a controlled design chain. Two baselines bracket the design space: Standard (no temporal information) and Additive (global time vector, no token-time binding). Joint (token and time projected jointly before self-attention) is a direct ablation of Timeformer that removes only the memory component. The full Timeformer extends Joint with causal prototype memory. All models are trained with masked language modeling on a synthetic benchmark with known ground-truth trajectories.
The main finding is that token-time interaction is the effective traceability mechanism. The Joint variant produces a neighborhood drift of -0.614 for drifting subjects, versus -0.231 for Standard (stable across seeds). Probe-based accuracy gains are modest and seed-sensitive, confirming that geometric diagnostics are more informative than label recoverability. The Memory-Augmented variant shows no robust improvement; oracle analysis identifies mean-prototype aggregation as the failure point.

Keywords: Temporal semantics · Semantic change · Transformer models · Traceability · Diagnostic evaluation

1 Introduction

Words do not preserve fixed meanings across time. In 1900, *broadcast* referred to scattering seeds across a field; today it describes a social media post reaching millions of users. Most language models, however, take no account of when a text was written: a word's representation depends on its surrounding context, but not on the historical period in which it appeared.

The ability to query a word at a specific period matters beyond technical benchmarks. Consider *gay*, which in the early twentieth century was associated with *cheerful*, *merry*, and *carefree*, and by the late twentieth century with *queer*, *lesbian*, and *bisexual* [3]. Whether this transition involved a shift toward or away from terms linked to social stigma is an empirical question that requires a

representation exposing `gay@1920` and `gay@1980` as directly comparable objects in a shared space.

The dominant computational approach, lexical semantic change detection (LSCD), measures such shifts by training distributional representations on time-sliced corpora and comparing them across periods [6, 7]. One model may associate *broadcast* in 1900 with *sow* and *field*, while another associates it with *signal* and *audience*; change is inferred from the distance between the two spaces. It summarizes the shift as a single similarity score between separately trained models, not as a queryable `token@time` object.

A subtler limitation arises when a single model receives temporal information during training. A model may correctly interpret *virus* as biological infection in 1900 and as malicious software in 2020, improving word-prediction accuracy, while still assigning nearly identical neighborhoods across periods. Prediction accuracy and geometric traceability are separable properties.

We call a representation *temporally traceable* if the same surface token, queried at different periods, occupies neighborhoods consistent with its meaning at that period, making `token@time` a directly inspectable object. This paper introduces Timeformer and evaluates it through a controlled design chain. Standard (no time information) and Additive (global time vector) serve as baselines. Joint (token and time projected jointly before self-attention) is a direct ablation of the full model that removes only the memory component. The full Timeformer extends Joint with causal prototype memory. This chain isolates which form of temporal conditioning produces traceability. We study this in a controlled synthetic benchmark in which the temporal trajectory of each subject is known, giving diagnostic access to target neighborhoods without the confounds of natural historical corpora.

The paper makes four contributions: it formulates temporal traceability as a `token@time` representation problem; introduces the Timeformer design chain; evaluates traceability with diagnostics that inspect subject representations directly (context drift score, contrastive sign-flip rate, and probe-based evaluation); and shows that token-time interaction is the main positive mechanism, while mean-pooled historical memory is better understood as a diagnostic extension. Section 2 reviews related work; Sections 3–5 describe the benchmark, architecture, and evaluation protocol; Section 6 reports results; Sections 7–8 discuss and conclude.

2 Related Work

Computational approaches to semantic change are commonly framed as lexical semantic change detection: given language data from different periods, the goal is to determine whether and how the meaning of a target item has changed. Diachronic word embeddings provided one of the most influential formulations of this problem by learning distributional representations for different time slices and comparing them across periods [3]. This family of methods made semantic change operational through geometric movement in embedding space, often af-

ter alignment or normalization across independently trained temporal slices. The central intuition remains important for this paper: semantic change should be visible as a change in neighborhood structure. Our focus, however, is not on aligning independently trained spaces. Instead, we ask whether a single Transformer-style encoder can expose a representation that is directly queryable as a lexical item at a specific period.

Benchmarks for LSCD have helped standardize evaluation by defining target words, periods, and gold annotations for semantic change [6, 7]. These resources are essential for evaluating change in natural corpora, but they inherit the observational complexity of historical language, where changes in frequency, genre, and corpus composition interact with lexical meaning. We use a controlled benchmark as a diagnostic testbed rather than a replacement for natural LSCD benchmarks, trading observational realism for direct access to planted trajectories and evaluation labels.

A second relevant line of work concerns temporal conditioning in neural models. Rather than training separate models for each period, a temporally conditioned model can receive time as part of its input or architecture, allowing parameter sharing across periods while representations or predictions vary with time [1, 5, 4]. For the traceability problem, however, the form of conditioning matters. A global time vector may tell the model which period it is processing, but it does not necessarily force an interaction between a particular token identity and that period. Timeformer is therefore evaluated through a controlled design chain that separates global time injection from explicit token-time interaction. This distinction is important because the paper evaluates not only whether time improves a prediction, but whether the same token occupies different and interpretable neighborhoods at different periods.

Finally, this paper draws on diagnostic evaluation and multi-sense representations. Probe accuracy does not establish traceability, so we combine it with geometric diagnostics that inspect the subject token directly. Semantic change may involve bifurcation rather than a simple shift [2]; the memory-augmented variant exposes why mean-pooled prototypes are insufficient for that structure.

3 Controlled Temporal Benchmark

The benchmark gives each subject a known temporal trajectory, making it possible to ask directly whether a model's representation of a subject at $t0$ has moved toward the expected neighborhood by $t9$. This distinguishes it from natural-language LSCD benchmarks, where ground truth is often annotated or inferred post-hoc and corpus confounds are hard to separate from genuine semantic change.

The corpus uses a subject–verb–object (SVO) grammar: every sentence consists of one subject, one verb, and one object. Two semantic neighborhoods, N_1 and N_2, partition the vocabulary of 8 verbs and 8 objects: N_1 uses $\{V1, \ldots, V4\}$ and $\{O1, \ldots, O4\}$; N_2 uses $\{V5, \ldots, V8\}$ and $\{O5, \ldots, O8\}$. A sentence generated from N_1 therefore looks like `S5 V2 O3`, while one from N_2 looks like `S5`

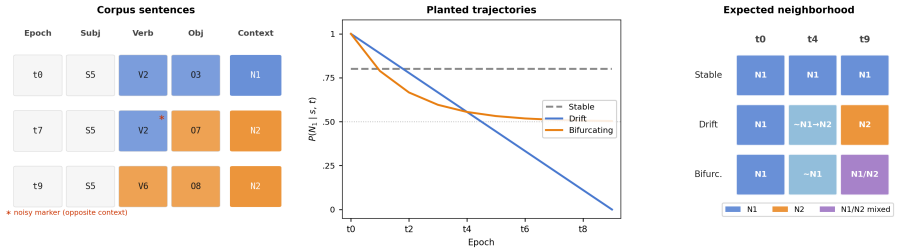

Fig. 1. Benchmark design overview. *Left*: SVO sentences for a drifting subject across three periods; an N_1 verb appearing in an N_2 sentence at $t7$ (marked $*$) illustrates the deliberate noise in local markers. *Center*: planted $P(N_1 \mid s, t)$ trajectories for Stable (flat), Drift (monotonic), and Bifurcating (levelling at 0.5) subjects across $t0$–$t9$. *Right*: expected nearest-neighbor behavior for a traceable representation.

`V6 O8`; the subject token is the same in both cases, but its co-occurring words differ. Intuitively, the intended meaning of subject s at period t is how often it appears in N_1 sentences; formally, $P(N_1 \mid s, t)$ is the probability that a sentence is drawn from N_1. The vocabulary contains 30 subjects; each subject's trajectory is determined by how $P(N_1 \mid s, t)$ evolves across the 10 periods $t0$ through $t9$.

Figure 1 (center) shows the three planted trajectory classes. Stable subjects maintain an approximately constant $P(N_1 \mid s, t)$ throughout. Drift subjects follow a monotonic trajectory from near-exclusive N_1 ($P(N_1 \mid s, t) \approx 1$ at $t0$) to near-exclusive N_2 ($P(N_1 \mid s, t) \approx 0$ at $t9$): a drifting subject that appears in sentences like `S5 V2 O3` early on will appear in sentences like `S5 V6 O8` by the final period. Bifurcating subjects begin in N_1 and move toward a mixed state ($P(N_1 \mid s, t) \approx 0.5$) where both neighborhoods appear with equal frequency, approximating two coexisting senses. The benchmark contains 10 subjects per class.

At period $t7$, for example, a sentence for `S5` might read `S5 V2 O7` (Figure 1, left): the planted neighborhood is N_2, but `V2` belongs to N_1. Each local marker is drawn from the opposite neighborhood 25% of the time, so the model cannot recover the planted neighborhood from the words alone. It must use subject identity together with the period. The `true_context` label is determined by the planted trajectory and is withheld from training; it is used only for evaluation.

The benchmark supports several evaluation conditions. The standard split uses the same 75% marker accuracy as training. Two harder splits force the verb, or both verb and object, to come from the opposite context. An ambiguous split sets fidelity to 50%, so local markers carry no reliable signal; only subject identity and period remain. Two additional conditions test temporal behaviour directly and are described in Section 5: the contrastive set presents the same sentence under two different period labels to check whether changing only the period changes the preferred context; the continuation set uses periods held out from training to test generalisation. Results throughout are diagnostic evidence in a controlled setting, not a claim about open-domain historical language.

4 Timeformer Architecture

Timeformer is designed to make a subject representation depend explicitly on the period in which the subject is queried. The full model adds two mechanisms to a standard Transformer encoder. First, it binds token identity and period at the input layer. Second, it lets the current subject consult causal historical prototypes before the encoder. We introduce these mechanisms through a controlled design comparison: two baselines (Standard and Additive), one direct ablation (Token-Time Transformer, labelled Joint in Figure 2), and the full Timeformer.

All models follow the same outer pipeline. A sentence is tokenized as `[CLS] S V O [SEP]`. Each token w_i is mapped to an input embedding $z_i \in \mathbb{R}^d$. The sequence $(z_1, \ldots, z_n)$ is processed by a Transformer encoder, and the final hidden state at the subject position,

$$(h_1, \ldots, h_n) = \mathrm{Encoder}(z_1, \ldots, z_n), \qquad h_s = h_{\mathrm{subj}},$$

is the `token@time` representation inspected by the diagnostics. All models are trained with masked language modeling (MLM): some tokens are masked, and the model must reconstruct them from the remaining tokens and, when available, the period input. What changes across the design comparison is how the subject's input embedding z_s is built before the encoder sees the sentence.

The temporally informed models use a shared period vector $\tau(t) \in \mathbb{R}^d$. It is a learned projection of sinusoidal features of t/T, where T is the total number of training periods. In plain terms, $\tau(t)$ is the vector representation of the current period. It is distinct from the position encoding p_i, which only tells the Transformer whether a token is in subject, verb, or object position. Additive and Joint differ in how they combine $\tau(t)$ with token identity; the full Timeformer then adds causal memory on top of Joint. Figure 2 summarizes the comparison.

4.1 Baseline: Standard Transformer

The Standard baseline asks whether temporal traceability appears without any period input. Its input embedding is the usual sum of a token embedding and a position encoding:

$$z_i = e(w_i) + p_i.$$

For the subject in `S5 V2 O3`, this gives $z_s = e(\texttt{S5}) + p_1$. Because no period signal enters the model, the same sentence at $t0$ and $t9$ has the same input sequence and therefore the same h_s. Standard is the null hypothesis: any temporal behavior must come only from changes in the observed words, not from an explicit `token@time` representation.

4.2 Baseline: Additive Time-Conditioned Transformer

The Additive baseline asks whether knowing the current period is enough. It adds the same period vector $\tau(t)$ to every token:

Figure 2. Controlled design comparison

Fig. 2. Controlled design comparison. **Standard**: Transformer baseline with no temporal signal. **Additive**: temporal baseline that adds the same period vector to every token. **Joint**: direct ablation of Timeformer without memory, using joint token-time projection. **Timeformer**: full model, extending Joint with Prototype Memory and Temporal Attention. New components are highlighted in yellow; dashed grey boxes mark inactive components.

$$z_i = e(w_i) + p_i + \tau(t).$$

Now `S5 V2 O3` at $t0$ and $t9$ are distinguishable. The subject embedding is $e(\texttt{S5}) + p_1 + \tau(0)$ at $t0$, and $e(\texttt{S5}) + p_1 + \tau(9)$ at $t9$. However, the same temporal change is also applied to the verb and object: `V2` receives $e(\texttt{V2}) + p_2 + \tau(t)$, and `O3` receives $e(\texttt{O3}) + p_3 + \tau(t)$. Thus Additive tells the whole sentence which period it belongs to, but it does not explicitly bind period to each token's identity. If Additive succeeds, global period information is sufficient. If it does not, the next question is whether the period must interact with each token before contextualization.

4.3 Ablation: Token-Time Transformer

The Token-Time Transformer, labelled Joint in Figure 2, is the full Timeformer without the memory component. It asks whether explicit token-time interaction is enough. Instead of adding the same period vector to all tokens, Joint concatenates each token embedding with $\tau(t)$ and projects the pair:

$$z_i = W[e(w_i); \tau(t)] + p_i,$$

where $e(w_i), \tau(t) \in \mathbb{R}^d$ and $W \in \mathbb{R}^{d \times 2d}$. The difference from Additive is visible in the example. At $t9$, the period vector is combined separately with `S5`, `V2`,

and O3. The model computes $W[e(\texttt{S5}); \tau(9)]$, $W[e(\texttt{V2}); \tau(9)]$, and $W[e(\texttt{O3}); \tau(9)]$ as separate inputs. The same period can therefore affect the subject differently from the verb or object. For traceability, the key case is the subject: S5 at $t0$ receives $W[e(\texttt{S5}); \tau(0)] + p_1$, while S5 at $t9$ receives $W[e(\texttt{S5}); \tau(9)] + p_1$. The subject starts from an epoch-specific input before self-attention, making $h_s(\texttt{S5}, t0)$ and $h_s(\texttt{S5}, t9)$ free to move in different semantic directions.

4.4 Full Model: Timeformer

The full Timeformer asks whether causal history helps beyond token-time interaction. It starts exactly like Joint: for a current sentence containing S5 at $t7$, the model first builds token-time input embeddings for `[CLS] S5 V O [SEP]`. It then modifies only the subject embedding before the Transformer encoder sees the sentence.

The additional information comes from Prototype Memory. Timeformer stores one prototype vector per subject and past period. A prototype $m(s, t)$ is the average representation of subject s across training sentences from period t. The memory is causal: at $t7$, the model may consult $\{m(\texttt{S5}, 0), \ldots, m(\texttt{S5}, 6)\}$, but not prototypes from $t7$ or later.

Temporal attention turns this list of past prototypes into a history summary for the current subject. Let z_s be the current Joint subject embedding and let M_s be the matrix of available causal prototypes, one row per past period. The attention operation uses z_s as the query and M_s as keys and values:

$$r_s = \mathrm{Attn}(z_s, M_s, M_s).$$

The vector r_s summarizes the parts of the subject's history that are most relevant to the current period. A learned scalar gate g then decides how much this history summary should change the current subject embedding:

$$\tilde{z}_s = z_s + g \cdot (\mathrm{LN}(z_s + r_s) - z_s)\,.$$

Here, LN denotes layer normalization. The gate g is initialized at zero. At initialization, $\tilde{z}_s = z_s$, so the full Timeformer behaves like Joint. During training, the model can learn to let memory affect the subject representation if doing so helps the MLM objective. Finally, $\tilde{z}_s$ replaces z_s in the input sequence, and the Transformer encoder produces the final representation h_s. This is the only place where Timeformer uses history: the verb and object embeddings are unchanged, but the subject enters the encoder already informed by its available causal prototype history.

5 Evaluation Protocol

All variants are trained with masked language modeling over the synthetic SVO corpus. The planted `true_context` label is never used during training; it is used only after training to evaluate whether the learned representation contains

temporal semantic information. This separation reflects the intended use case of representation learning from corpora without semantic-change labels. The primary evaluation target is h_s, the final hidden state at the subject position, since subjects are the lexical items with planted temporal trajectories.

We use four complementary diagnostics. First, a linear probe is trained on h_s from the standard test split to predict `true_context` and is then evaluated on each split. This measures whether context information is linearly recoverable from the representation, but it does not by itself establish traceability. Second, we compute a context drift score over nearest neighbors. For each period and subject class, we retrieve the k nearest neighbors of each subject representation and measure the proportion assigned to N_1, with k fixed at the same value across all models and splits. A traceable representation should show a strong movement from N_1-like to N_2-like neighborhoods for drifting subjects, weaker movement for bifurcating subjects, and comparatively limited movement for stable subjects. Third, we use contrastive temporal examples in which the same subject token is presented with two different period labels. We define the contrastive sign-flip rate as the proportion of such pairs for which swapping the period label reverses the nearest-neighbor assignment (from N_1-dominant to N_2-dominant or vice versa). A model without temporal capacity is expected to produce a sign-flip rate of zero, since the same token yields the same representation regardless of the period label provided. Fourth, we evaluate continuation examples from held-out later periods to test whether temporal behavior extends beyond the training distribution.

Comparisons follow the design chain. Additive vs. Standard isolates the effect of global time conditioning. Joint vs. Additive isolates explicit token-time interaction at the input layer. Timeformer vs. Joint isolates the memory contribution, evaluated on continuation and contrastive diagnostics.

For the memory-augmented model, we include a shuffled-memory control (a subject receives prototypes from a different subject), a no-history control (the memory path is present but empty), and an oracle-memory diagnostic (the model receives perfect historical prototypes). Memory is considered useful only when it improves over the Token-Time Transformer and over both inappropriate and empty histories.

6 Results

We report results over three training runs. Unless otherwise noted, values are means over seeds, with the standard deviation shown in parentheses when space allows. The main finding is that token-time interaction produces a much clearer temporal movement in representation space than the Standard Transformer. The memory-augmented variant is informative as a diagnostic extension, but it does not provide a robust improvement over the Token-Time Transformer.

Fig. 3. Context drift score across periods. The Token-Time Transformer shows stronger movement from N_1 toward N_2 for drifting subjects than the Standard Transformer, while stable and bifurcating subjects provide class-specific controls.

Table 1. Context drift score at the first and last periods. Values are N_1 proportions among nearest neighbors; $\Delta = t9 - t0$.

Class	Model	t0	t9	Δ
Stable	Standard	0.736	0.756	+0.020
	Joint	0.843	0.641	-0.203
	Timeformer	0.810	0.658	-0.153
Drift	Standard	0.618	0.387	-0.231
	Joint	0.847	0.233	-0.614
	Timeformer	0.836	0.263	-0.573
Bifurcation	Standard	0.794	0.682	-0.112
	Joint	0.895	0.521	-0.374
	Timeformer	0.905	0.517	-0.388

6.1 Temporal Neighborhood Movement

Figure 3 and Table 1 show the context drift score across periods. For drifting subjects, the Standard Transformer drops from 0.618 to 0.387 ($\Delta = -0.231$), while the Token-Time Transformer shows a much stronger transition from 0.847 to 0.233 ($\Delta = -0.614$). This is the clearest evidence for temporal traceability: the same subject token occupies substantially different neighborhoods when queried at different periods.

The class-specific curves serve as controls (Table 1). Stable subjects show much smaller movement and no coherent A-to-B trend. Bifurcating subjects show intermediate drift consistent with a trajectory toward a mixed state, not a complete transition. The geometric diagnostic thereby distinguishes all three trajectory classes without relying on a single aggregate score.

Table 2. Main traceability diagnostics. Accuracy columns report linear-probe accuracy on h_s; flip is contrastive sign-flip rate. A dash indicates the metric was not computed for this variant.

Model	Drift	Flip	Ambig.	Cont.
Standard	-0.231	0.000	0.577	0.719
Additive	–	0.690	0.573	0.754
Joint	-0.614	0.517	0.599	0.766
Timeformer	-0.573	0.506	0.599	0.766

6.2 Probe and Contrastive Diagnostics

The linear probe results are supportive but less decisive than the neighborhood analysis. On the ambiguous split, where local verb and object markers are uninformative, the Standard Transformer reaches 0.577 accuracy, the Additive Time-Conditioned Transformer reaches 0.573, and the Token-Time Transformer reaches 0.599. The corresponding mean delta for token-time interaction is +0.026, but it is seed-sensitive: the three runs give +0.053, +0.005, and +0.021. This confirms that probe accuracy alone should not carry the central claim. It measures recoverability of the planted label, not the geometry of temporal movement.

The contrastive sign-flip diagnostic gives a different view. The Standard Transformer has a sign-flip rate of 0.000, as expected. The Additive variant averages 0.690 while the Token-Time Transformer averages 0.517, an ordering that may appear counterintuitive. The decoupling arises because sign-flip counts threshold crossings in individual contrastive pairs, whereas drift score measures the full trajectory magnitude. A global additive offset displaces all tokens uniformly, generating many crossings without coherent subject-level movement. Token-time interaction produces more structured, subject-specific movement (evidenced by the stronger drift) but does not necessarily cross the threshold in more pairs. We therefore treat sign flip as evidence that time conditioning changes behavior at all, and the neighborhood trajectories as the more diagnostic measure of traceability.

6.3 Memory Diagnostics

The Memory-Augmented Timeformer does not produce a robust gain over the Token-Time Transformer. On continuation, both models average 0.766, and the per-seed memory deltas are +0.013, +0.005, and −0.018. The oracle diagnostic clarifies the failure mode: oracle memory improves contrastive sign flip (0.586 vs. 0.448 for the Token-Time reference) but not continuation (0.754 vs. 0.762). Perfect historical prototypes can affect contrastive temporal behavior, but do not solve the continuation objective under the current MLM setup, supporting the use of Timeformer as a diagnostic extension.

Table 3. Memory diagnostics from the corrected oracle run. Continuation is linear-probe accuracy on held-out later periods; flip is contrastive sign-flip rate.

Variant	Cont.	Flip	Δ vs. Joint	
Joint (reference)	0.762	0.448	–	
Timeformer learned	0.775	0.448	+0.013 +0.000 flip	cont.;
Timeformer oracle	0.754	0.586	−0.008 +0.138 flip	cont.;

7 Discussion

The results support a clear conclusion: temporal traceability is a geometric property that does not reduce to predictive accuracy. The Token-Time Transformer improves probe accuracy by only +0.026 on average (modest and seed-sensitive), yet shows a neighborhood drift of −0.614 vs. −0.231 for the Standard baseline. This dissociation is the central empirical finding: a model can use period information to improve predictions without exposing a representation in which the same lexical item occupies meaningfully different neighborhoods at different periods. Evaluating traceability therefore requires geometric diagnostics, not only label recoverability.

The additive variant shows that knowing the period is not sufficient. A global additive offset shifts all token representations by the same vector, leaving subject-level trajectory geometry weakly constrained. Token-time interaction changes this: concatenating each token embedding with the time encoding before projection allows the same period to affect different tokens differently, so the same subject receives a distinct embedding at each period. This is the mechanism behind the coherent neighborhood trajectories in the drift analysis.

The Memory-Augmented Timeformer tests the simplest form of causal memory: a mean of historical subject representations per period. Learned prototypes produce near-zero average gain over the Token-Time Transformer, and the oracle diagnostic identifies why: perfect historical prototypes improve contrastive sign-flip rate but not continuation accuracy. For bifurcating subjects, which develop two coexisting senses in later periods, the mean prototype is a centroid between two clusters, representing neither sense well. This motivates sense-aware memory mechanisms that preserve the internal structure of historical representations rather than collapsing it, and constitutes the main open direction from this work.

8 Conclusion

This paper studied whether a Transformer-style encoder can expose a temporally traceable representation, meaning one in which the same lexical item, queried at different periods, occupies neighborhoods that reflect its intended temporal context. The answer, in the controlled benchmark used here, is conditional:

explicit token-time interaction is necessary. A Standard encoder and a globally time-conditioned encoder both fail to produce coherent subject-level trajectories. An encoder that conditions each token embedding on time before self-attention is applied does produce such trajectories, with a neighborhood drift for drifting subjects that is more than twice that of the Standard baseline (-0.614 vs. -0.231).

The contribution is narrow and deliberate: token-time interaction supports traceability in a controlled diagnostic setting, not a claim about open-domain historical language. The evaluation protocol also contributes a method for separating representational traceability from predictive performance. Mean-pooled historical prototypes do not produce a robust improvement, and the oracle diagnostic identifies single-prototype aggregation as the failure point. Sense-aware memory mechanisms are the natural next step.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Dhingra, B., Cole, J.R., Eisenschlos, J.M., Gillick, D., Eisenstein, J., Cohen, W.W.: Time-aware language models as temporal knowledge bases. Transactions of the Association for Computational Linguistics (TACL) **10**, 257–273 (2022)
2. Giulianelli, M., Del Tredici, M., Fernández, R.: Analysing lexical semantic change with contextualized word representations. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL). pp. 3960–3973 (2020)
3. Hamilton, W.L., Leskovec, J., Jurafsky, D.: Diachronic word embeddings reveal statistical laws of semantic change. In: Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL). pp. 1489–1501 (2016)
4. Loureiro, D., Barbieri, F., Neves, L., Anke, L.E., Camacho-Collados, J.: TimeLMs: Diachronic language models from Twitter. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL): System Demonstrations. pp. 251–260 (2022)
5. Rosin, G., Radinsky, K.: Time masking for temporal language models. In: Proceedings of the 15th ACM International Conference on Web Search and Data Mining (WSDM). pp. 833–841 (2022)
6. Schlechtweg, D., McGillivray, B., Hengchen, S., Dubossarsky, H., Tahmasebi, N.: SemEval-2020 task 1: Unsupervised lexical semantic change detection. In: Proceedings of the 14th International Workshop on Semantic Evaluation (SemEval). pp. 1–23 (2020)
7. Tahmasebi, N., Borin, L., Jatowt, A.: Survey of computational approaches to lexical semantic change. Computational Approaches to Semantic Change pp. 1–91 (2021)